Intermediate DevOps
Day 3: Containers, Kubernetes and continuous delivery

United Airlines · Pluralsight instructor-led training

Nick Lotz

27–29 October 2026

Day 2 recap

- Infrastructure as code describes the desired state in version-controlled files.
- Terraform compares the configuration with its state and plans the changes.
- The `platform` repository runs a plan on every pull request and applies on merge.
- Terraform state is stored in a Kubernetes Secret. A Lease stops two runs from applying at once.
- Credentials get the least privilege and the shortest lifetime that still work. A pipeline never prints them.

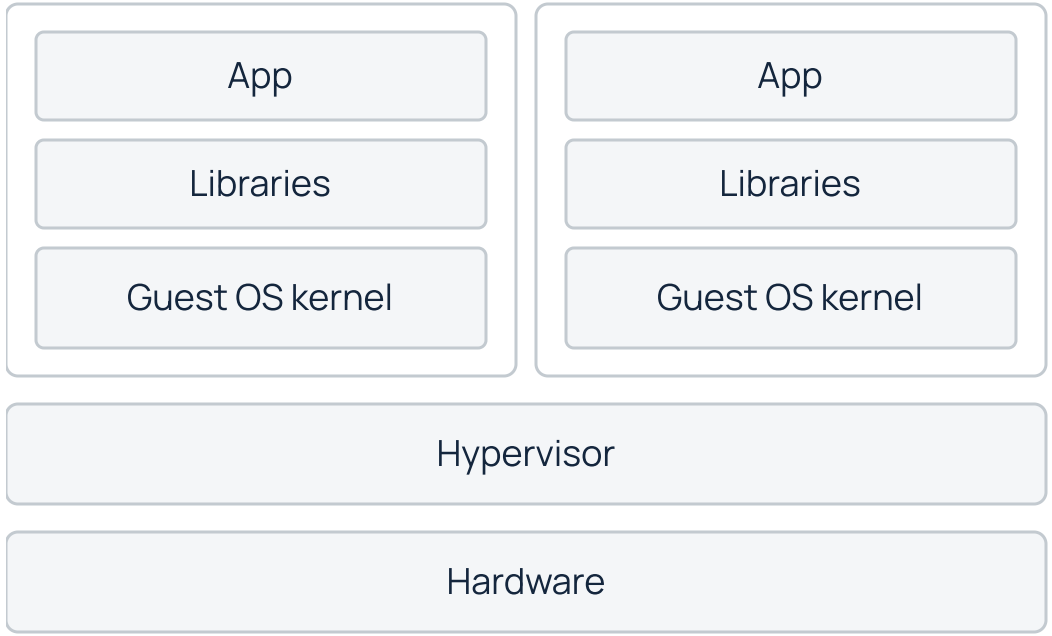
Your cluster now has a dev and a prod namespace, each with a quota and a Postgres database.

Day 3 agenda

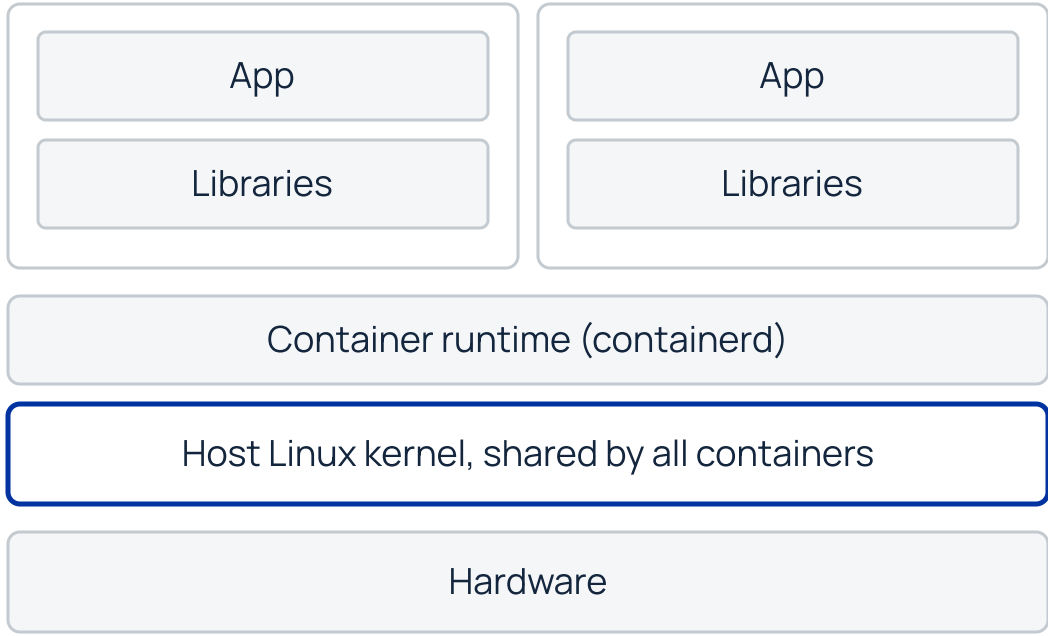
Time (CT)	Session
09:00	Day 2 recap
09:10	Containers
09:45	Docker Compose, Amazon ECS and Fargate
10:10	Break
10:20	Lab 07: Build, scan and describe a container
11:35	Kubernetes
12:10	Lunch
12:55	Lab 08: Kubernetes deployments, rollouts and rollbacks
14:25	Break
14:35	Continuous delivery
15:05	Lab 09: End-to-end delivery pipeline
16:50	Course summary and feedback

Virtual machines and containers

Virtual machines



Containers



A virtual machine runs its own kernel on virtual hardware. A container is a group of ordinary processes on the host kernel, with an isolated view of the system.

Linux namespaces isolate what a process can see

Namespace	What the process gets its own copy of
PID	Process IDs. The container's first process is PID 1.
Network	Network interfaces, IP addresses, routes and ports
Mount	The filesystem tree, built from the image
UTS	The hostname
IPC	Shared memory and message queues
User	User and group IDs, so root in a container can map to an unprivileged host user
Cgroup	The view of its own control group

Docker creates a new set of namespaces for each container (namespaces(7), Linux manual pages).

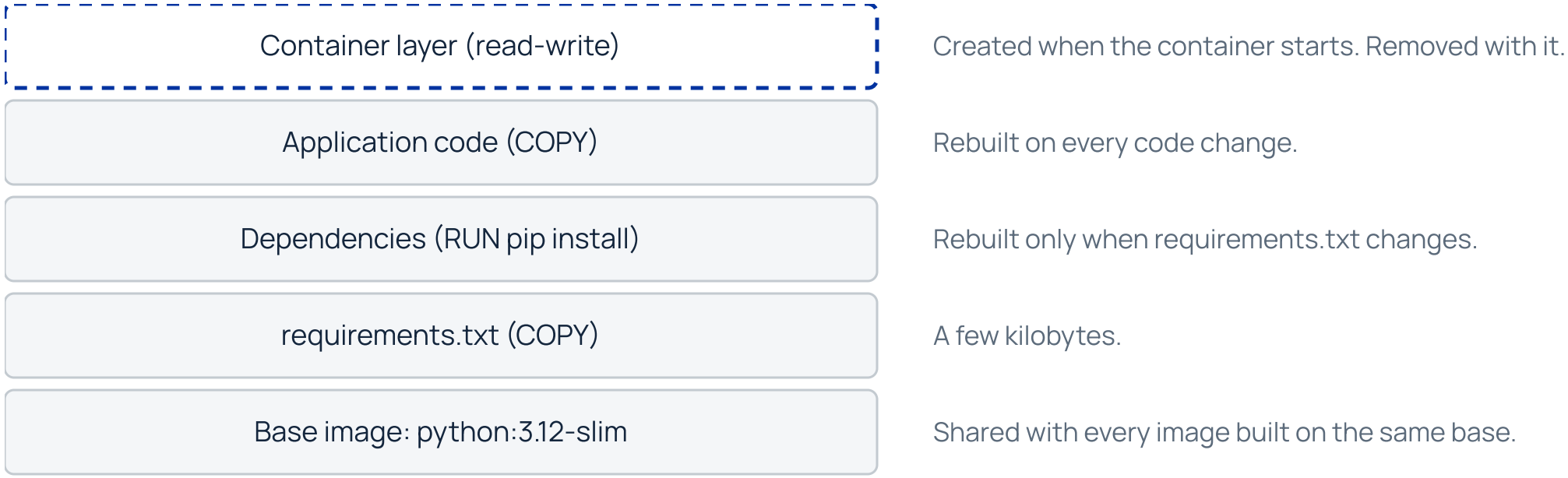
Control groups limit what a process can use

- A control group (cgroup) limits the CPU time, memory, disk I/O and number of processes of a group of processes.
- The kernel also counts usage per cgroup. `docker stats` reads these counters.
- When a container uses more memory than its limit, the kernel kills a process in it (OOM kill).
- When a container reaches its CPU limit, the kernel slows it down (throttling). It keeps running.

```
docker run --memory 256m --cpus 0.5 flight-status:1.0.0
```

Kubernetes requests and limits, later today, are applied through the same cgroup settings.

Images are stacks of read-only layers



Each Dockerfile instruction that changes files adds a read-only layer. OverlayFS stacks the layers into one filesystem and adds a writable layer for each container.

The OCI specifications

Specification	What it defines	Examples
Image	The image manifest, configuration and layer format	Docker BuildKit, Buildah
Runtime	How to run a container from an unpacked filesystem and a configuration file	runc, crun
Distribution	The HTTP API for pushing and pulling images	Docker Hub, Amazon ECR, the k3d registry

The Open Container Initiative was founded in 2015 to standardise these formats. Because of them, an image built with Docker on your lab machine runs on containerd in k3d, on Amazon EKS and on Amazon ECS.

Tags and digests

```
k3d-registry.localhost:5000/flight-status:1.4.0
k3d-registry.localhost:5000/flight-status@sha256:9b1e5c0d…
```

- A tag is a name that points to an image. Anyone with push rights can point it at a different image.
- A digest is the SHA-256 hash of the image manifest. It identifies one image and cannot change.
- `latest` is the default tag name. It does not mean the newest image.
- Deploy by digest, or by a tag that your pipeline never reuses.

Dockerfile practices

- Start from a small, maintained base image, for example `python:3.12-slim` instead of `python:3.12`.
- Copy the dependency list and install it before you copy the source code.
- Run the process as a non-root user.
- Keep the build context small with `.dockerignore`, and exclude `.git`, caches and local secrets.
- Run one main process per container and write logs to standard output.

A Dockerfile for a Python web service

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY flightstatus/ ./flightstatus/
RUN useradd --uid 10001 --no-create-home app
USER 10001
EXPOSE 8000
CMD ["uvicorn", "flightstatus.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- Dependencies are installed before the code is copied, so a code change reuses the cached dependency layer.
- The server runs as user 10001, which has no special rights on the host.

Container risks in NIST SP 800-190

Area	Example risk	What this course does about it
Image	Known vulnerabilities in packages; secrets stored in a layer	Trivy scan in the pipeline; secret scanning on push
Registry	Stale or untrusted images; unencrypted connections	Deploy by unique tag or digest; TLS for real registries such as ECR
Orchestrator	Too much access for people and pipelines	A namespaced ServiceAccount with RBAC
Container	Running as root; no resource limits	Non-root user; requests and limits
Host OS	One kernel shared by all containers	Patched hosts, or no hosts to manage with Fargate

NIST SP 800-190 groups container risks into these five areas and gives countermeasures for each (Souppaya et al., 2017).

Software bills of materials and vulnerability scanning

- A software bill of materials (SBOM) lists every package in an image, with its version. CycloneDX and SPDX are the common formats.
- A scanner such as Trivy matches those packages against vulnerability databases.
- Each finding has a severity. A pipeline can fail when it finds a CRITICAL vulnerability that has a fix.
- New vulnerabilities are published every day, so an image that passed last month can fail today.

Signing and provenance

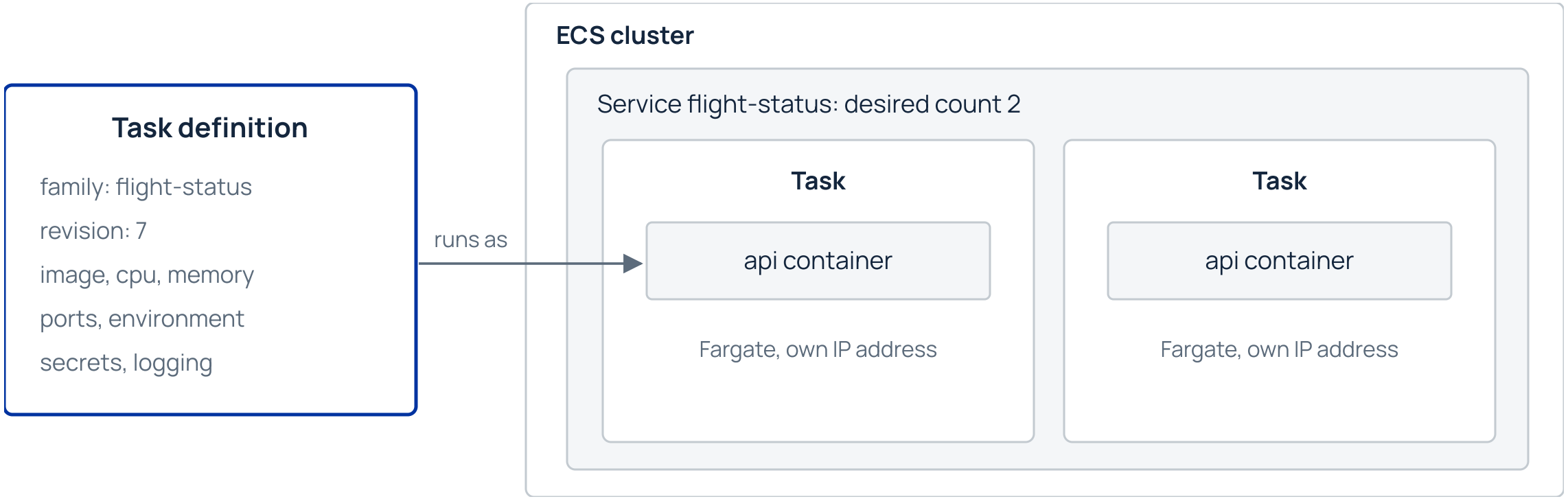
- A signature shows who published an image and that it has not changed since.
- Sigstore's cosign signs an image digest and stores the signature in the registry next to the image.
- Provenance records how an image was built, including the source commit, the build system and the build steps.
- SLSA Build levels 1 to 3 ask for provenance, then provenance signed by a hosted build platform, then a hardened build platform.
- A cluster admission policy can refuse images that are not signed by a trusted key.

Docker Compose describes a multi-container application

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://flights:flights@db:5432/flights
    depends_on:
      - db
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: flights
      POSTGRES_PASSWORD: flights
      POSTGRES_DB: flights
```

Compose creates one network for the project, and each service is reachable by its name. The API reaches the database at the host name `db`.

Amazon ECS concepts



A task definition is a versioned JSON document. A task is one running copy of it, and a service keeps the desired number of tasks running.

Fargate runs tasks without servers to manage

	EC2 launch type	Fargate
Who manages the hosts	You, as a group of EC2 instances	AWS
What you size	Instances, then the tasks placed on them	The CPU and memory of each task
Network mode	bridge, host or awsvpc	awsvpc only, so each task gets its own network interface
Billing	Per instance	Per task, for vCPU and memory, per second
Access to the host	Possible	None

At United, containers run on Amazon ECS with Fargate.

Mapping Compose to an ECS task definition

compose.yaml	Task definition (JSON)
`image:`	`containerDefinitions[].image`
`ports: ["8000:8000"]`	`portMappings: [{"containerPort": 8000}]`
`environment:`	`environment: [{"name": ..., "value": ...}]`
A password in `environment:`	`secrets: [{"name": ..., "valueFrom": "<Secrets Manager ARN>"}]`
`healthcheck:`	`healthCheck.command`
`deploy.resources.limits`	Task-level `cpu` and `memory`
Logs on standard output	`logConfiguration` with the `awslogs` driver
`depends_on:`	`dependsOn` inside one task; separate services otherwise

What changes when the task runs in AWS

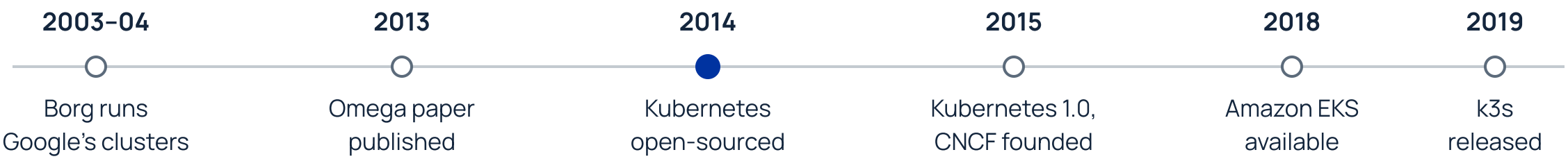
- Each Fargate task gets its own network interface and private IP address in a VPC subnet.
- The task execution role lets ECS pull the image and read secrets. The task role is what the application uses to call AWS APIs.
- An Application Load Balancer sends requests to healthy tasks. It replaces the port mapping on your machine.
- The database is a managed service such as Amazon RDS.
- Logs go to Amazon CloudWatch Logs.

Lab 07: Build, scan and describe a container

75 minutes. Open `labs/07-containers.md` in the course repository.

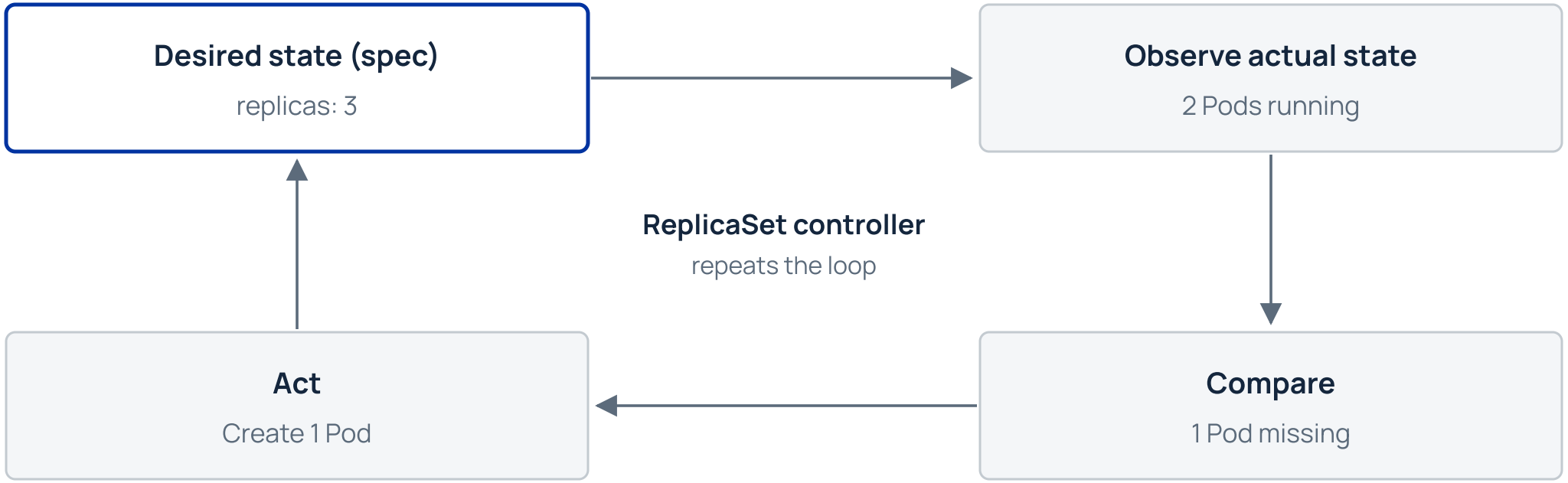
- Write a Dockerfile, build the flight-status image and inspect its layers.
- Push the image to the local registry.
- Install Trivy, scan the image and produce an SBOM.
- Run the API with Postgres in Docker Compose, then map the Compose file to an ECS task definition.

From Borg to Kubernetes



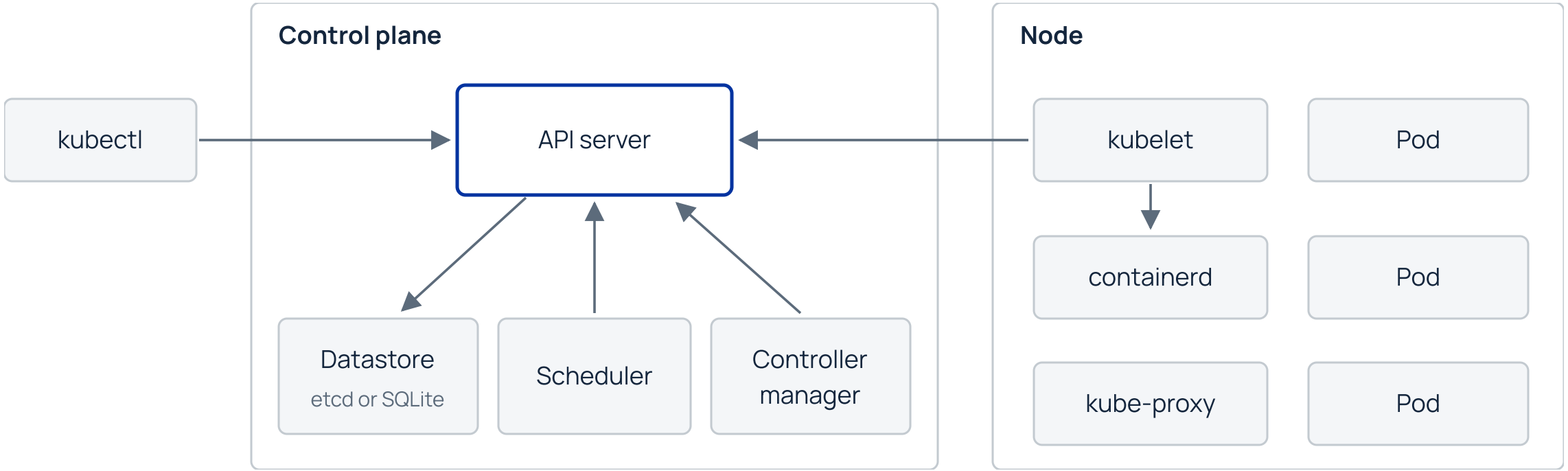
Burns et al. (2016) describe what Google learned from Borg and Omega. Kubernetes adopted labels instead of numbered jobs, one IP address per Pod, and independent controllers that reconcile state.

The declarative API and reconciliation loops



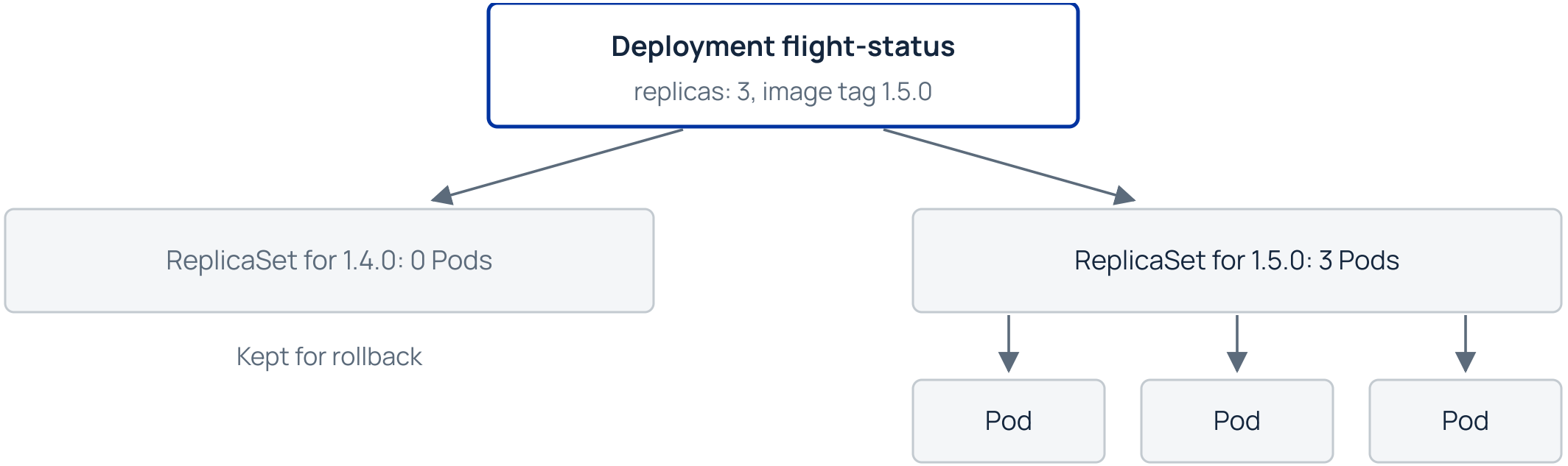
Every Kubernetes controller runs this loop. A deleted Pod comes back because the ReplicaSet controller sees two Pods where the spec asks for three, and creates one.

The control plane and the nodes



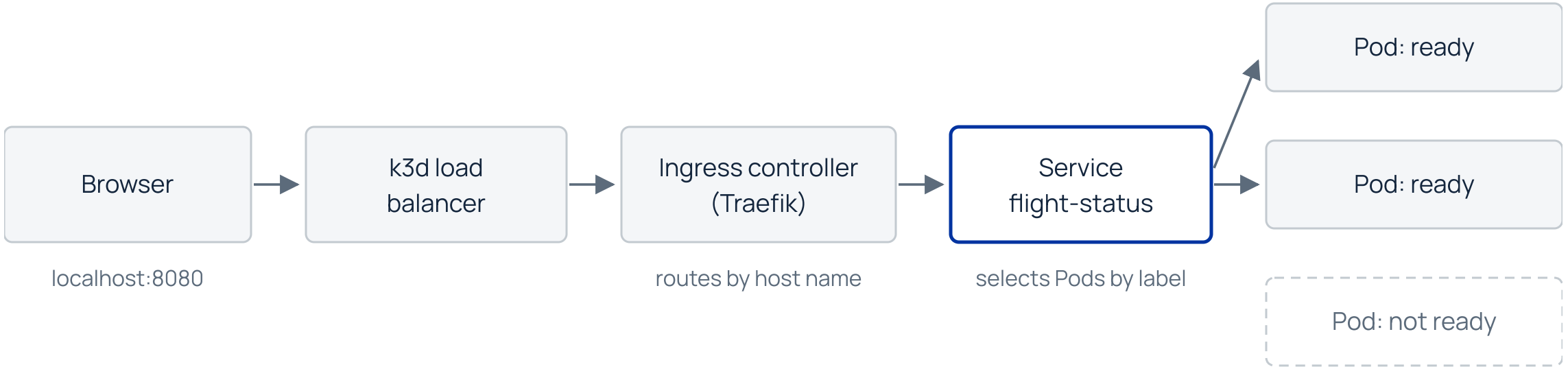
Every component talks to the API server, and only the API server uses the datastore. In k3d, one container (`k3d-devops-server-0`) runs the control plane and your Pods.

Deployments, ReplicaSets and Pods



A Pod is one or more containers that share an IP address and volumes. A Deployment creates a new ReplicaSet for each new Pod template, scales the old one to zero and keeps it for rollback.

Services and Ingress



A Service gives a set of Pods one stable name and address, and sends traffic only to Pods that pass their readiness probe. An Ingress routes HTTP requests to Services by host name and path.

Liveness, readiness and startup probes

Probe	Question it answers	What happens when it fails
Startup	Has the application finished starting?	The other probes wait. After too many failures, the container restarts.
Liveness	Is the process still able to work?	The kubelet restarts the container.
Readiness	Can this Pod take requests now?	The Pod is removed from the Service. It is not restarted.

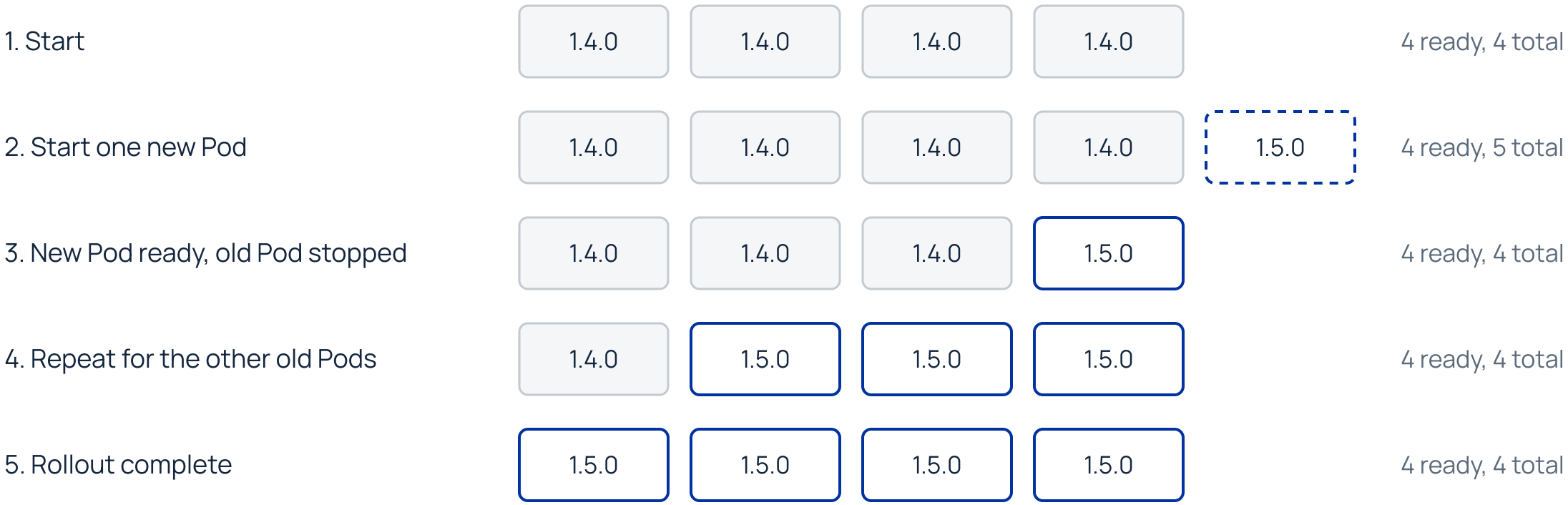
The flight-status API answers liveness checks on `/health` and readiness checks on `/ready`, which also tests the database connection.

Requests, limits and quality of service

- A request is the amount the scheduler reserves on a node. A Pod only goes to a node with enough unreserved capacity.
- A limit is the most a container may use. Over its memory limit it is killed. At its CPU limit it is throttled.

QoS class	Rule	Under memory pressure on the node
Guaranteed	Every container has CPU and memory requests equal to its limits	Evicted last
Burstable	At least one request or limit, but not Guaranteed	Evicted by how far use exceeds requests
BestEffort	No requests or limits	Evicted first

A rolling update, step by step



Settings: 4 replicas, maxSurge 1, maxUnavailable 0. There are never fewer than 4 ready Pods or more than 5 Pods. The dashed Pod has started but is not ready yet.

Role-based access control



A Role allows verbs on resource types in one namespace, and a RoleBinding grants the Role to a user, group or ServiceAccount. RBAC rules only add permissions, so anything a Role does not list is refused.

k3d and Amazon EKS

	k3d on your lab machine	Amazon EKS
Control plane	A k3s server in one Docker container	Run by AWS across availability zones
Nodes	The same container	EC2 managed node groups or Fargate
Ingress	Traefik, bundled with k3s	AWS Load Balancer Controller with an ALB
Registry	`k3d-registry.localhost:5000` over HTTP	Amazon ECR over HTTPS with IAM
Workload identity	ServiceAccount tokens	ServiceAccount tokens plus IAM roles (EKS Pod Identity)

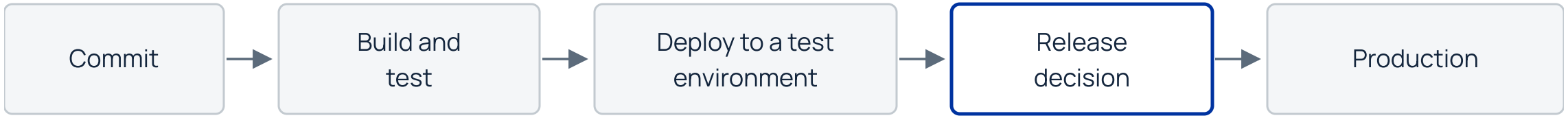
The Kubernetes API, kubectl, the manifests and RBAC are the same in both. That is why the Lab 08 manifests would run on EKS with only the image and Ingress settings changed.

Lab 08: Kubernetes deployments, rollouts and rollbacks

90 minutes. Open `labs/08-kubernetes-rollouts.md` in the course repository.

- Deploy the API with a Deployment, Service and Ingress, including probes, requests and limits.
- Scale it, delete a Pod and watch Kubernetes replace it.
- Roll out a new version, then a broken image and a version that fails its readiness probe.
- Roll back and read the rollout history.

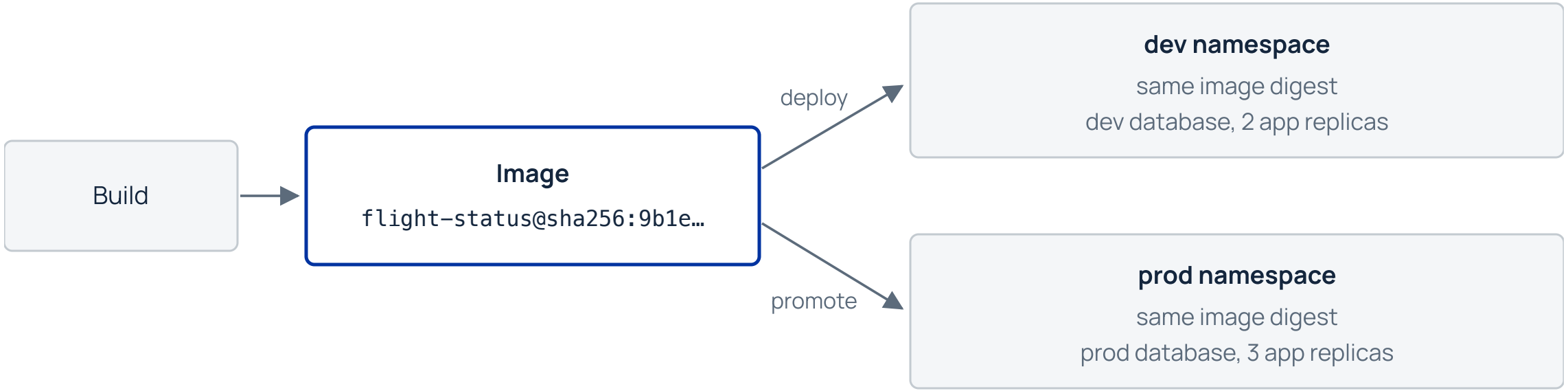
Continuous delivery and continuous deployment



Delivery: a person decides.
Deployment: the pipeline decides.

Continuous delivery keeps every change that passes the pipeline ready to release at any time (Humble and Farley, 2010). Continuous deployment releases each of those changes automatically.

Build once, promote the same artifact



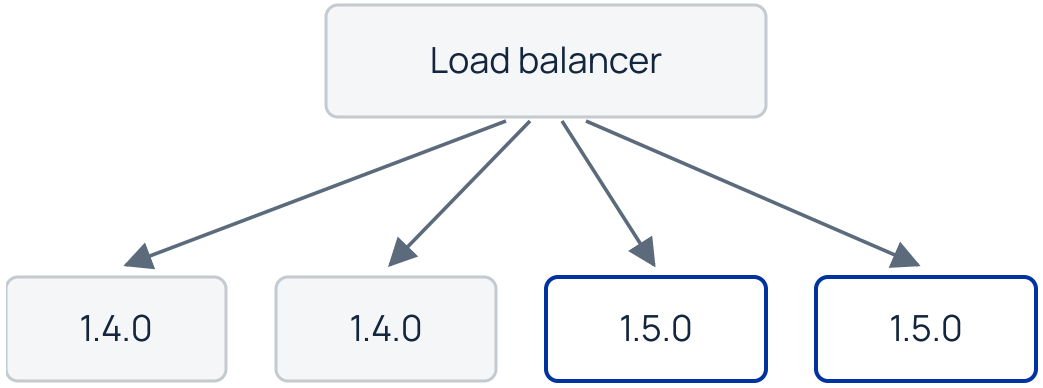
The image that passed the checks in dev is the image that runs in prod. Settings that differ between environments come from the environment, as variables and Secrets (Wiggins, 2011).

Approvals

- An approval is a pipeline step that waits for a named person or group to confirm.
- Use one where a person adds information the pipeline does not have, such as a change window or a business sign-off.
- Every approval adds waiting time to the lead time for changes.
- The Harness platform has Harness, Jira and ServiceNow approval steps, with timeouts.
- Harness Open Source has no approval step. In Lab 09, a person promotes a release to prod by pushing a git tag.

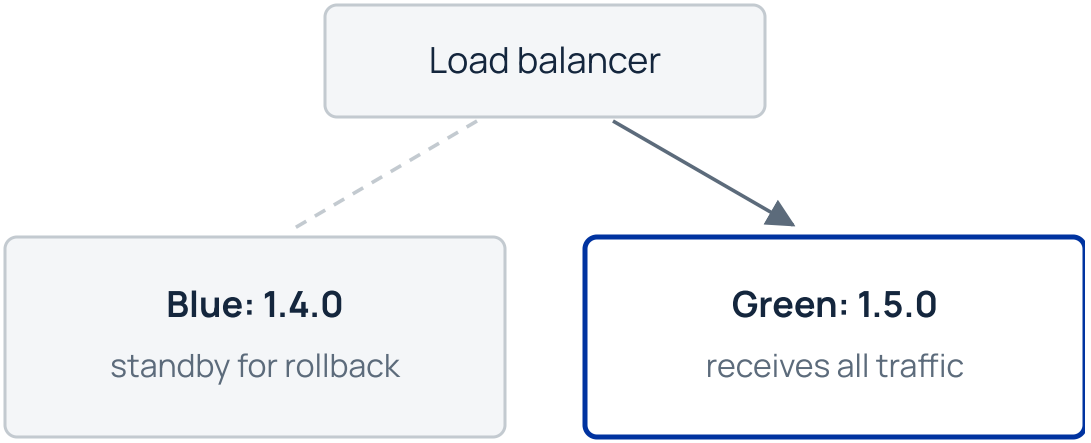
Rolling and blue-green deployments

Rolling



Old and new versions serve traffic during the rollout.

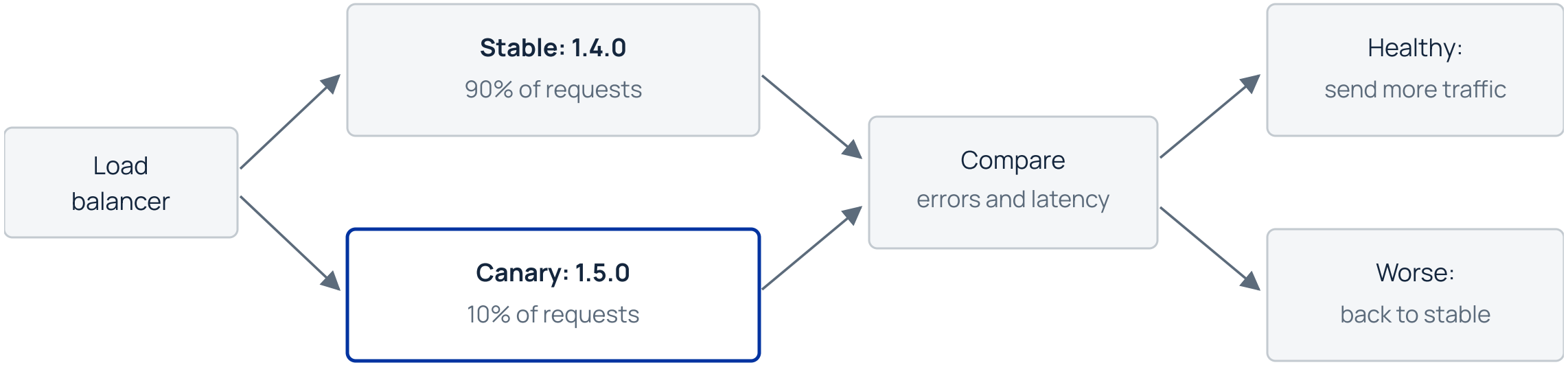
Blue-green



Traffic moves to the new environment in one switch.

A rolling deployment needs little extra capacity but runs two versions side by side for a while. Blue-green needs two full environments and rolls back by switching traffic back (Fowler, 2010).

Canary releases



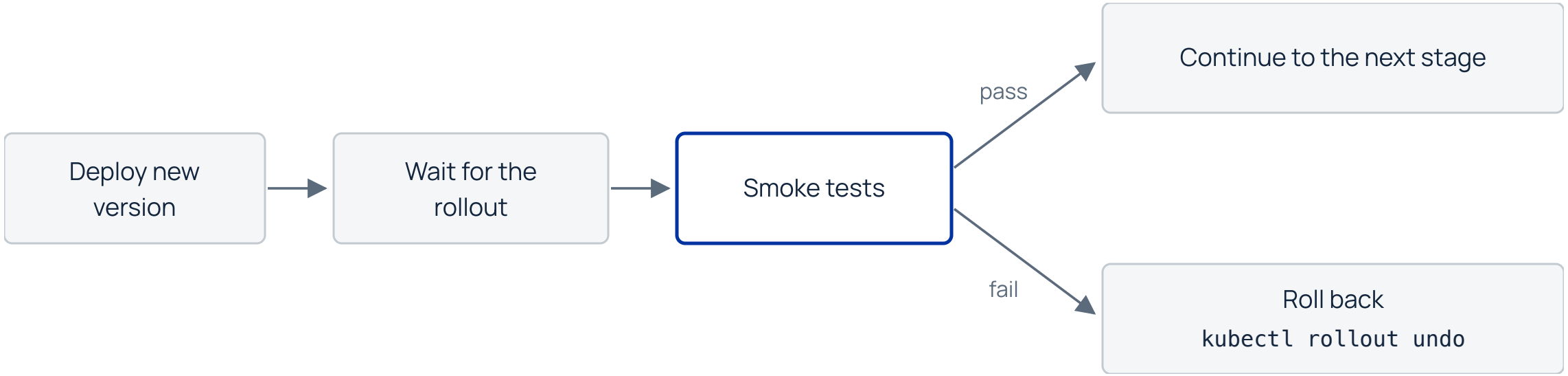
A canary release sends a small share of real traffic to the new version and compares it with the current version before sending more (Sato, 2014). With plain Kubernetes, the split follows the Pod count.

Feature flags separate deployment from release

```
if flags.is_enabled("new-departures-board", user):
    return new_departures_board(flights)
return departures_board(flights)
```

- The new code is deployed to production, switched off.
- Turning the flag on releases the feature to some users or to everyone, without a deployment.
- Turning it off again is the fastest rollback for that feature.
- Each flag needs an owner and a removal date. Old flags multiply the code paths you must test (Hodgson, 2017).

Automated verification and rollback



A smoke test calls the new version the way a user would: a health check, a version check, one real request. The Lab 09 smoke test checks the version through the ingress; a "Going further" task adds the other two. A step with `when: failure()` rolls back when any earlier step fails.

Lab 09 steps and the Harness platform

Lab 09 step in Harness Open Source	Harness platform feature for the same job
Run step with pytest and a Postgres background step	Build stage with a Run step and a Background step
Docker plugin step that builds and pushes the image	Build and Push to Docker step with a registry connector
Trivy scan step	Security Testing Orchestration with the Aqua Trivy step
kubectl step with a kubeconfig secret	Deploy stage: service, environment, infrastructure definition, Kubernetes connector and delegate
Smoke test step	Verify step (Continuous Verification) or a Shell Script step
Rollback step with `when: failure()`	Failure strategy with the Rollback Stage action
Promote pipeline started by a release tag	Approval stage, then a Deploy stage for the prod environment

DORA metrics revisited

Metric	Practices from this course that improve it
Deployment frequency	Small pull requests, an automated pipeline, one image promoted through environments
Lead time for changes	A pipeline on every push, fast tests, few manual approvals
Change failure rate	Tests and quality gates, image scanning, readiness probes, canary releases
Failed deployment recovery time	Immutable images, kept ReplicaSets, automated rollback

Forsgren et al. (2018) found that teams with high delivery speed also had high stability. The same practices improved both.

Lab 09: End-to-end delivery pipeline

105 minutes. Open `labs/09-end-to-end-delivery.md` in the course repository.

- Build one pipeline: test, build, scan, push, deploy to dev and smoke test.
- Promote the same image to prod by pushing a release tag.
- Release a broken version on purpose and watch the pipeline roll it back.
- Compare each step with its Harness platform feature.

What the three days covered

Day	Concepts	Labs
1	DevOps foundations, continuous integration, pipelines in Harness, testing and quality gates	01–03
2	Infrastructure as code, Terraform, infrastructure in pipelines, credential management	04–06
3	Containers, Compose, ECS and Fargate, Kubernetes, continuous delivery	07–09

Your Harness project now tests, builds, scans and deploys the flight-status API to two environments that you defined as code.

Next steps and feedback

- Keep the course repository. Every lab runs again on a new Ubuntu 24.04 machine.
- Open one of your team's pipelines in the Harness platform and find each stage from the Lab 09 mapping table.
- Pluralsight has paths on Docker, Kubernetes, Terraform and AWS for more depth on each topic.
- Please fill in the course survey. The next version of this course uses your answers.

References

- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue* 14(1).
- Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate*. IT Revolution.
- Fowler, M. (2010). BlueGreenDeployment. martinfowler.com
- Hodgson, P. (2017). Feature Toggles (aka Feature Flags). martinfowler.com
- Humble, J., and Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.
- Sato, D. (2014). CanaryRelease. martinfowler.com
- Souppaya, M., Morello, J., and Scarfone, K. (2017). *Application Container Security Guide*. NIST SP 800-190.
- Wiggins, A. (2011). *The Twelve-Factor App*. 12factor.net
- Open Container Initiative. Image Format, Runtime and Distribution Specifications. opencontainers.org
- SLSA (2023). Supply-chain Levels for Software Artifacts, v1.0. slsa.dev
- Linux manual pages: namespaces(7), cgroups(7). man7.org
- Kubernetes documentation: Deployments, probes, resource management, RBAC. kubernetes.io/docs
- Amazon Web Services. Amazon ECS Developer Guide: task definitions and Fargate. docs.aws.amazon.com
- DORA (2024). Accelerate State of DevOps Report. dora.dev
- Harness documentation: CI, CD, STO and failure strategies. developer.harness.io